
Numpy

NumPy (Numerical Python) is a Python library used for:

- Numerical computation
- Array processing
- Mathematical operations
- Matrix operations
- Statistical calculations
- Data manipulation

Arrays

- Import NumPy using:

import numpy as np

- **NumPy Arrays**

An array is a collection of elements of the same data type arranged in one or more dimensions.

- **1D Array**

a = np.array([10, 20, 30, 40])

- Representation:

[10 20 30 40]

- **2D Array**

```
a = np.array([[10, 20],
              [30, 40]])
```

- Representation:

10 20

30 40

- **3D Array**

A 3D array contains multiple 2D arrays.

```
a = np.array([[[1, 2], [3, 4]],
              [[5, 6], [7, 8]]])
```

Consider:

```
A = np.array([[10, 20, 30],
              [40, 50, 60]])
```

- **Number of dimensions**

```
print(A.ndim)
```

Output:

2

- **Shape**

```
print(A.shape)
```

Output:

(2, 3)

This means:

2 rows × 3 columns

◈ **Number of elements**

print(A.size)

Output:

6

◈ **Data type**

print(A.dtype)

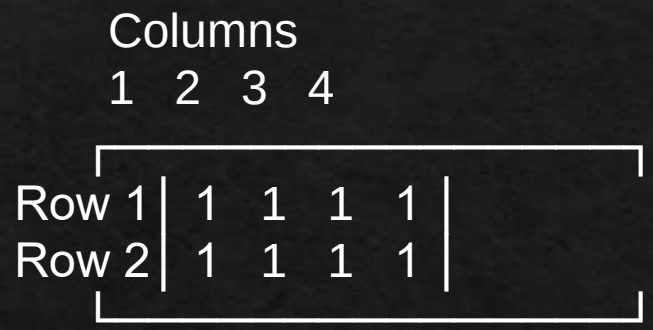
- **Creating Arrays**
- **Zeros**

np.zeros((3, 3))

```
[[0. 0. 0.]
[0. 0. 0.]
[0. 0. 0.]]
```

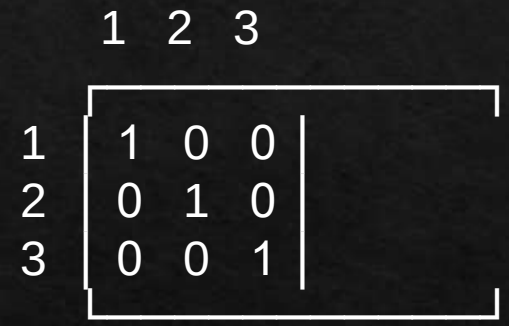
- **Ones**

np.ones((2, 4))



- **Identity matrix**

np.eye(3)



- **Range**

np.arange(1, 11)

Output:

[1 2 3 4 5 6 7 8 9 10]

- **Indexing and Slicing**

Consider:

```
A = np.array([10, 20, 30, 40, 50])
```

- Access an element:

```
print(A[0])
```

10

- Access multiple elements:

```
print(A[1:4])
```

[20 30 40]

- For a matrix:

```
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(A[0, 1])
```

2

- **Mathematical Operations**

Consider:

A = np.array([10, 20, 30])

B = np.array([2, 4, 5])

- **Addition**

A + B

[12 24 35]

- **Subtraction**

A - B

[8 16 25]

- **Multiplication**

A * B

[20 80 150]

- **Division**

A / B

[5,5,6]

- **Power**

A ** 2

[100 400 900]

- NumPy provides mathematical functions such as:
- np.sqrt(A)
- np.square(A)
- np.exp(A)
- np.log(A)
- np.sin(A)
- np.cos(A)
- np.abs(A)
- Example:

```
A = np.array([1, 4, 9, 16])

print(np.sqrt(A))
```

Output:

```
[1. 2. 3. 4.]
```

- **Statistical Operations**

Suppose:

marks = np.array([70, 80, 90, 60, 85])

- **Sum**

np.sum(marks)

- **Mean**

np.mean(marks)

- **Median**

np.median(marks)

- **Maximum**

np.max(marks)

- **Minimum**

np.min(marks)

- **Standard deviation**

np.std(marks)

- **Variance**

np.var(marks)

- These operations are particularly useful when **analysing datasets**.

- **Matrix Operations**
- Consider:

```
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])
```

- **Matrix addition**

```
A + B
```

- **Matrix subtraction**

```
A - B
```

- **Element-wise multiplication**

```
A * B
```

- This gives:

```
[[ 5 12]
 [21 32]]
```

- **Matrix multiplication**

A @ B

or:

np.matmul(A, B)

- Output:

[[19 22]
 [43 50]]

- **Transpose**

A.T

- **Determinant**

np.linalg.det(A)

- **Inverse**

np.linalg.inv(A)

◈ Data Manipulation Using NumPy

Data manipulation means modifying or reorganizing data to make it suitable for analysis.

Important operations include:

◈ Reshaping

```
A = np.arange(1, 7)
B = A.reshape(2, 3)
print(B)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

◈ Flattening

```
B.flatten()
```

Converts a multidimensional array into a one-dimensional array.

- **Sorting**

```
A = np.array([50, 20, 40, 10, 30])
print(np.sort(A))
```

Output:

```
[10 20 30 40 50]
```

- **Concatenation**

```
A = np.array([1, 2, 3])
B = np.array([4, 5, 6])
print(np.concatenate((A, B)))
```

- **Filtering**

```
A = np.array([10, 25, 30, 45, 50])
print(A[A > 30])
```

Output:

```
[45 50]
```

- This is called **Boolean indexing**.

Analysing a Dataset Using NumPy

Student	Maths	Python	Networking
S1	80	75	85
S2	65	70	60
S3	90	88	95
S4	72	80	78
S5	85	92	88

```
import numpy as np

# Dataset
marks = np.array([
    [80, 75, 85],
    [65, 70, 60],
    [90, 88, 95],
    [72, 80, 78],
    [85, 92, 88]
])

print("Student Marks:")
print(marks)
```

```
# Basic properties
print("\nNumber of dimensions:", marks.ndim)
print("Shape:", marks.shape)
print("Number of elements:", marks.size)

# Total marks for each student
total = np.sum(marks, axis=1)
print("\nTotal marks:")
print(total)

# Average marks for each student
average = np.mean(marks, axis=1)
print("\nAverage marks:")
print(average)
```

```
# Subject-wise average
subject_average = np.mean(marks, axis=0)
print("\nSubject-wise average:")
print(subject_average)

# Highest and lowest marks
print("\nHighest mark:", np.max(marks))
print("Lowest mark:", np.min(marks))

# Highest mark in each subject
print("\nHighest mark in each subject:")
print(np.max(marks, axis=0))
```

```
# Lowest mark in each subject
print("\nLowest mark in each subject:")
print(np.min(marks, axis=0))

# Students with average above 80
print("\nAverages above 80:")
print(average[average > 80])

# Sorted averages
print("\nSorted averages:")
print(np.sort(average))
```